

Java Backend Architecture

Interview Series

Chapter 11.4

Distributed Locks

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

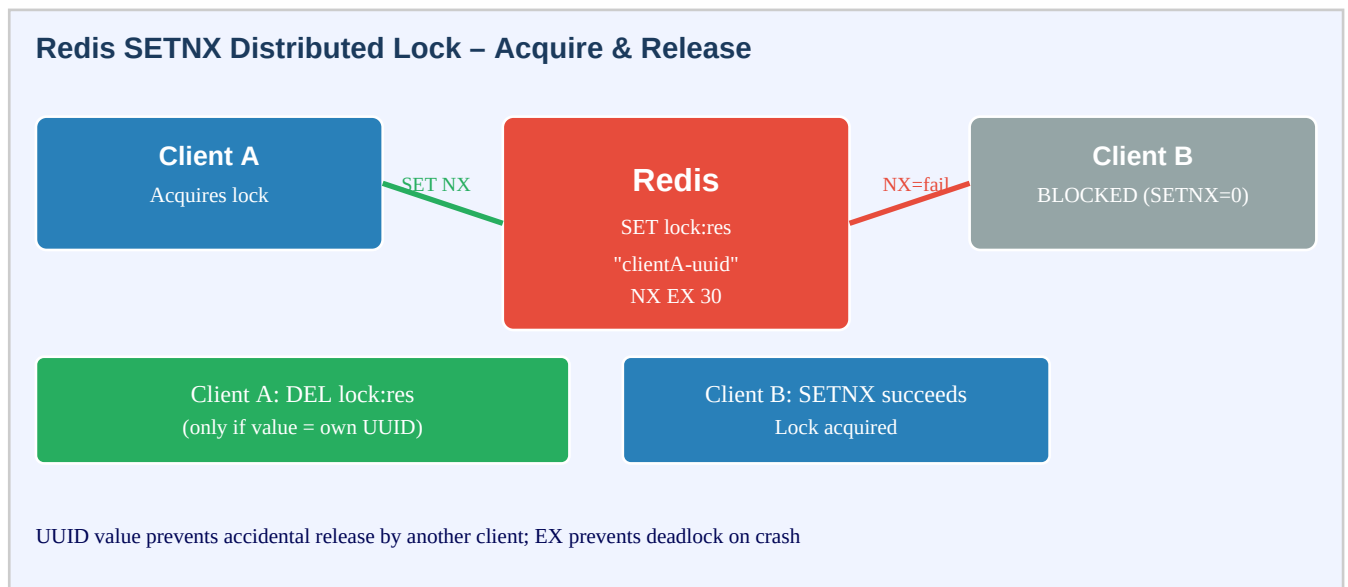
Chapter 11.4 – Distributed Locks

Introduction

Distributed locks coordinate access to a shared resource across multiple processes or nodes in a distributed system. Unlike single-process locks (synchronized, ReentrantLock), distributed locks must tolerate network partitions, process crashes, and clock skew. The fundamental challenge is that a lock holder can crash after acquiring the lock but before releasing it, potentially leaving the resource permanently locked — motivating TTL-based expiry. But TTL creates a race: what if the lock expires while the holder is still working (due to a GC pause or network delay)?

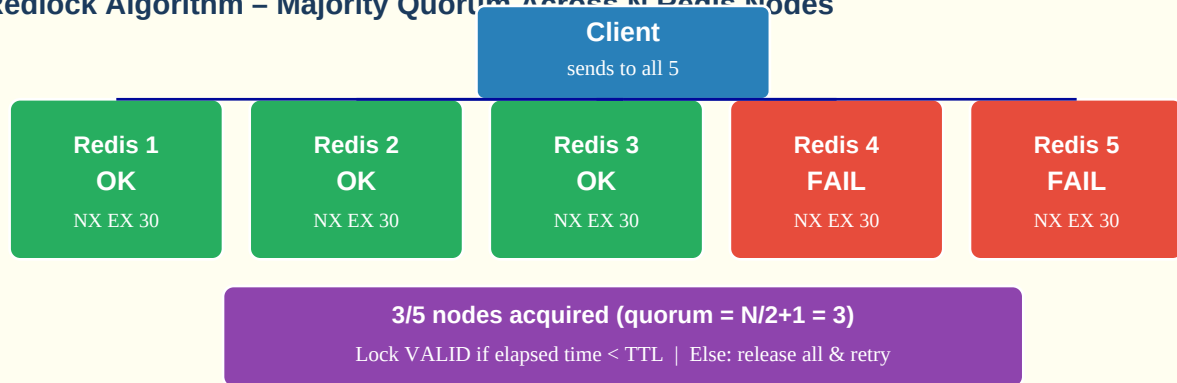
Redis is the most popular distributed lock backend in Java systems due to its low latency and atomic SET NX EX semantics. The Redlock algorithm extends this to multi-node Redis for higher availability. ZooKeeper ephemeral nodes offer stronger safety guarantees via sequential node ordering. PostgreSQL advisory locks provide database-integrated coordination. Redisson (a Java Redis client) provides a production-ready RLock with watchdog-based lease renewal. Understanding the tradeoffs — especially Martin Kleppmann's critique of Redlock — is essential for choosing the right tool.

Redis SETNX Distributed Lock



Redlock – Majority Quorum Algorithm

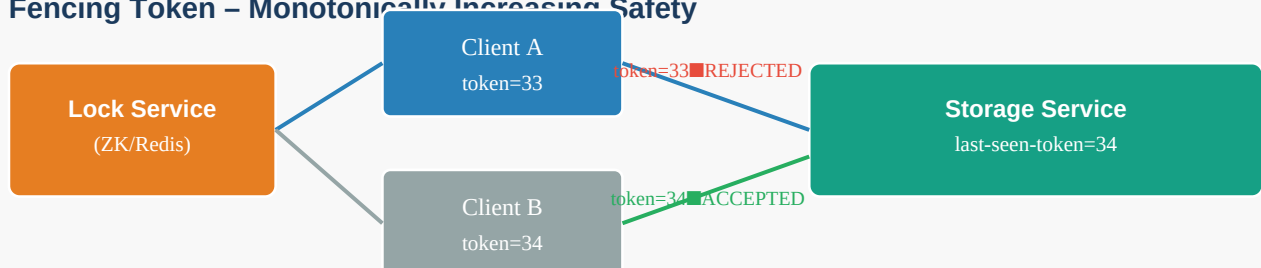
Redlock Algorithm – Majority Quorum Across N Redis Nodes



N=5, quorum=3: tolerate 2 node failures | Each SET uses unique random value

Fencing Tokens – Monotonically Increasing Safety

Fencing Token – Monotonically Increasing Safety



Storage rejects writes with token <= last-seen-token. Prevents stale lock holder from writing.

Martin Kleppmann proposes fencing tokens as safer alternative to Redlock.

Key Definitions

SETNX	SET if Not eXists — Redis command; atomic check-and-set used for lock acquisition
SET NX EX	Atomic Redis command: SET key value NX (only if absent) EX seconds (TTL); safer than SETNX+EXPIRE
Redlock	Redis algorithm acquiring locks on N independent Redis nodes; lock valid if acquired on $N/2+1$ nodes within TTL
Quorum	Majority of Redis nodes ($\text{floor}(N/2)+1$); ensures at most one client holds the lock at a time
Fencing token	Monotonically increasing integer issued with lock grant; storage rejects writes with stale tokens
Watchdog	Background thread that extends lock TTL while holder is still alive; prevents expiry under slow work

RLock	Redisson Java class implementing distributed reentrant lock with watchdog, try-lock, fair-lock
Ephemeral node	ZooKeeper node automatically deleted when the client session ends; used for lock and leader election
Sequential node	ZooKeeper node with auto-incremented suffix; lowest-numbered node among siblings holds the lock
Advisory lock	PostgreSQL pg_try_advisory_lock(key): application-level lock not tied to a specific row or table
Split-brain	Two nodes each believe they hold the lock simultaneously due to network partition
Lease	Lock with a bounded TTL; if not renewed before expiry, automatically released
Lock expiry	TTL-based automatic release; prevents permanent deadlock on crash but introduces race window
Crash safety	If lock holder crashes, TTL ensures the lock is eventually released automatically
Thundering herd	All waiting clients retry simultaneously when a lock is released; solved by randomized backoff

Distributed Lock Backends Comparison

Backend	Mechanism	Strength	Weakness
Redis SETNX	SET NX EX atomic	Fast (<1ms)	Single node SPOF; TTL race
Redlock	Majority N nodes	No SPOF	Kleppmann critique; complex
ZooKeeper	Ephemeral sequential	Strong safety, watch API	Slower, heavier
PostgreSQL Advisory	pg_advisory_lock()	DB-integrated, transactions	DB load, no TTL
Redisson RLock	Redis + watchdog	Java-native, lease renewal	Redis dependency

Code Examples

1. Redis distributed lock with Jedis (SET NX EX + Lua delete)

```
@Component public class RedisDistributedLock { private final JedisPool jedisPool; private
static final String LOCK_PREFIX = "lock:"; /** * Acquire lock. Returns unique token if
acquired, null if not. */ public String tryAcquire(String resource, int ttlSeconds) { String
token = UUID.randomUUID().toString(); try (Jedis jedis = jedisPool.getResource()) { // SET key
token NX EX seconds – atomic, no SETNX+EXPIRE race String result = jedis.set( LOCK_PREFIX +
resource, token, SetParams.setParams().nx().ex(ttlSeconds) ); return "OK".equals(result) ?
token : null; } } /** * Release lock only if we still own it (Lua script for atomicity). */
public boolean release(String resource, String token) { // Lua: compare-and-delete atomically –
prevents releasing another client's lock String luaScript = "if redis.call('get', KEYS[1]) ==
ARGV[1] then " + " return redis.call('del', KEYS[1]) " + "else " + " return 0 " + "end"; try
(Jedis jedis = jedisPool.getResource()) { Object result = jedis.eval(luaScript,
Collections.singletonList(LOCK_PREFIX + resource), Collections.singletonList(token)); return
Long.valueOf(1).equals(result); } } } // Usage: String token = lock.tryAcquire("order:payment",
30); if (token == null) throw new LockNotAcquiredException("payment already processing"); try {
processPayment(); } finally { lock.release("order:payment", token); }
```

2. Redisson RLock with watchdog (lease renewal)

```
@Configuration public class RedissonConfig { @Bean public RedissonClient redissonClient() {
    Config config = new Config(); config.useSingleServer().setAddress("redis://localhost:6379");
    // Default lock watchdog timeout = 30 seconds // Watchdog renews TTL every 10 seconds while
    holder is alive config.setLockWatchdogTimeout(30_000); return Redisson.create(config); } }
@Service public class PaymentService { private final RedissonClient redisson; public void
processPayment(String orderId) throws InterruptedException { RLock lock =
redisson.getLock("lock:payment:" + orderId); // tryLock: wait up to 5s to acquire, hold for 30s
(renewed by watchdog) boolean acquired = lock.tryLock(5, 30, TimeUnit.SECONDS); if (!acquired)
{ throw new LockNotAcquiredException("Payment " + orderId + " already processing"); } try { //
Watchdog automatically extends the 30s TTL every 10s // So even if payment takes 2 minutes,
lock won't expire prematurely performPaymentProcessing(orderId); } finally { if
(lock.isHeldByCurrentThread()) { lock.unlock(); } } } }
```

3. Redlock algorithm implementation sketch

```
// Conceptual Redlock with 5 independent Redis nodes (no replication between them) public class
RedlockClient { private final List<RedisDistributedLock> nodes; // 5 independent Redis
instances private static final int QUORUM = 3; // floor(5/2) + 1 public Optional<String>
acquire(String resource, int ttlMs) { String token = UUID.randomUUID().toString(); long start =
System.currentTimeMillis(); int acquired = 0; for (RedisDistributedLock node : nodes) { try {
String result = node.tryAcquire(resource, ttlMs / 1000); if (result != null) acquired++; }
catch (Exception ignored) { // Node unavailable – count as failure } } long elapsed =
System.currentTimeMillis() - start; long validFor = ttlMs - elapsed; if (acquired >= QUORUM &&
validFor > 0) { return Optional.of(token); } // Failed: release all acquired nodes
nodes.forEach(n -> { try { n.release(resource, token); } catch (Exception ignored) {} });
return Optional.empty(); } } // Kleppmann critique: even with quorum, a GC pause between
acquiring the lock // and using it can cause the lock to expire, leading to split-brain. //
Solution: fencing tokens (storage rejects stale token writes).
```

4. ZooKeeper ephemeral sequential node lock

```
// Using Apache Curator (ZooKeeper client library) @Component public class
ZooKeeperDistributedLock { private final CuratorFramework curator; public void withLock(String
lockPath, Runnable work) throws Exception { InterProcessMutex lock = new
InterProcessMutex(curator, "/locks/" + lockPath); // InterProcessMutex: creates ephemeral
sequential node under /locks/path // Acquires when node has the lowest sequence number among
siblings // ZooKeeper watch notifies when predecessor node is deleted if (lock.acquire(10,
TimeUnit.SECONDS)) { try { work.run(); } finally { lock.release(); // Session expiry
automatically removes ephemeral node → releases lock } } else { throw new
LockNotAcquiredException("Could not acquire ZK lock: " + lockPath); } } } // Key properties: //
- Ephemeral node: auto-deleted when client session expires (crash safety) // - Sequential
ordering: no thundering herd (only lowest-seq node is notified) // - Strong consistency:
ZooKeeper is CP (consistent, partition-tolerant) // - No TTL race: ZK session expiry is
separate from lock TTL // - Tradeoff: ZK adds latency (~1-5ms) vs Redis (~0.1ms)
```

5. PostgreSQL advisory lock

```
// PostgreSQL session-level advisory lock @Repository public class PgAdvisoryLockRepository {
private final JdbcTemplate jdbc; // Acquire lock (blocks until available) public void
acquireLock(long lockKey) { jdbc.execute("SELECT pg_advisory_lock(" + lockKey + ")"); } // Try
lock (returns true if acquired, false immediately if not) public boolean tryAcquireLock(long
lockKey) { return Boolean.TRUE.equals( jdbc.queryForObject( "SELECT pg_try_advisory_lock(?)",
```

```
Boolean.class, lockKey)); } public void releaseLock(long lockKey) { jdbc.execute("SELECT
pg_advisory_unlock(" + lockKey + ")"); } } // Transaction-level advisory lock (auto-released at
commit/rollback): // SELECT pg_advisory_xact_lock(hashtext('order-processing')); // This lock
is held for the duration of the transaction only. // Lock key generation from string: // long
key = Math.abs((long) "payment-service".hashCode()); // // Use case: DB-integrated coordination
where you already have a PG connection. // Advantage: participates in same transaction as
business data writes. // Limitation: no built-in TTL; connection drop releases the lock.
```

6. Fencing token storage-side enforcement

```
// Storage service enforces fencing token monotonicity @Service public class
SharedResourceService { private final AtomicLong lastSeenToken = new AtomicLong(-1); // Write
with fencing token – rejects stale lock holders public void writeData(String data, long
fencingToken) { long current = lastSeenToken.get(); if (fencingToken <= current) { throw new
StaleLockException( "Token " + fencingToken + " is stale; last seen: " + current); } // CAS
update to avoid race on lastSeenToken itself if (!lastSeenToken.compareAndSet(current,
fencingToken)) { throw new ConcurrentModificationException("Concurrent token update"); }
persistData(data); } } // Lock service issues monotonically increasing tokens: // Token 33
issued to Client A, Token 34 issued to Client B (after A's expiry) // If Client A (with stale
token 33) tries to write after Client B has written with 34: // → StaleLockException thrown →
Client A's write is rejected safely // // This is Kleppmann's solution: even if Redlock has a
split-brain moment, // the storage layer enforces correctness via token ordering.
```

Interview Questions & Answers

Q1. What is a distributed lock and why is it different from a JVM-level lock?

A distributed lock coordinates exclusive access across multiple processes running on different machines, whereas a JVM lock (synchronized, ReentrantLock) is scoped to a single JVM. Differences: (1) Network latency — acquiring/releasing a distributed lock requires network round-trips (0.1-5ms vs nanoseconds for JVM). (2) Crash failure — if a lock holder crashes, a JVM lock is automatically released (JVM terminates), but a distributed lock remains held until TTL expires or cleanup runs. (3) Clock skew — distributed systems have unreliable clocks, making TTL-based expiry imprecise. (4) Network partitions — a partitioned node may believe it holds a lock while another node has acquired the same lock (split-brain). These challenges require careful design: unique lock tokens, TTL with watchdog, and ideally fencing tokens.

Q2. Why must you use SET key value NX EX seconds instead of SETNX followed by EXPIRE?

SETNX + EXPIRE is a two-command sequence — not atomic. Between SETNX succeeding and EXPIRE being executed, the client can crash. The key is then set without a TTL, creating a permanent lock that nobody can acquire — a distributed deadlock. The atomic SET key value NX EX seconds command was added in Redis 2.6.12 to solve this: both the 'set if not exists' and the expiry are set in a single atomic command. Always use the single SET command with NX and EX parameters together. Never split them, even if the code looks cleaner.

Q3. Explain the Redlock algorithm.

Redlock acquires a distributed lock across N independent Redis instances (e.g., 5). Steps: (1) Record start time. (2) For each Redis node, attempt SET lock:resource uuid NX EX ttlMs (using a short per-node timeout to avoid blocking on slow nodes). (3) Count successes. (4) After all attempts, compute elapsed time. (5) If successes $\geq \text{floor}(N/2)+1$ (quorum) AND elapsed $< \text{TTL}$, the lock is acquired and valid for remaining TTL. (6) If quorum not reached or TTL exceeded, release on all nodes that succeeded and retry with backoff. The quorum ensures that even if $N/2$ nodes fail, the lock is held on a majority. The TTL remainder ensures the lock

doesn't expire before the holder can use it.

Q4. What is Martin Kleppmann's critique of Redlock?

Kleppmann argues that Redlock is unsafe even with a quorum because it relies on timing assumptions that can be violated: (1) A lock holder acquires Redlock with TTL=10s. (2) Due to a GC stop-the-world pause of 12 seconds, the lock expires on all Redis nodes. (3) Another client acquires the same lock. (4) The original client resumes from its pause, still believing it holds the lock, and modifies the shared resource concurrently with the new lock holder — split-brain. No TTL-based mechanism can be safe against arbitrary process pauses. The correct solution per Kleppmann is fencing tokens: the storage system must enforce monotonicity and reject writes from stale lock holders regardless of what the lock service says.

Q5. What is a fencing token and how does it solve the split-brain problem?

A fencing token is a monotonically increasing integer issued by the lock service each time a lock is granted (e.g., token 33, then 34 on the next grant). The lock holder includes its token in every write to the storage service. The storage service remembers the highest token it has seen and rejects any write with a token $\leq$ last-seen-token. If Client A holds expired lock (token=33) and Client B has the current lock (token=34), Client A's writes are rejected ($33 \leq 34$). Client B's writes succeed. This provides safety even in the presence of GC pauses, network delays, and Redlock split-brain scenarios. The correctness guarantee is in the storage layer, not the lock service.

Q6. How does Redisson's watchdog prevent premature lock expiry?

Redisson's RLock sets an initial TTL of 30 seconds (lockWatchdogTimeout). It starts a background watchdog thread that fires every TTL/3 (10 seconds) to check if the lock is still held by the current JVM thread. If yes, it extends the TTL by another 30 seconds via a Redis PEXPIRE command. This continues until lock.unlock() is called explicitly, which stops the watchdog and releases the lock. If the JVM crashes, the watchdog stops and the lock expires normally after 30 seconds. This balances safety (crash eventually releases the lock) with liveness (long-running work doesn't lose the lock). Never use a fixed TTL without watchdog for operations of uncertain duration.

Q7. How do ZooKeeper ephemeral nodes work as distributed locks?

ZooKeeper distributed lock using ephemeral sequential nodes: (1) All clients try to create an ephemeral sequential node under /locks/resource — e.g., /locks/resource/lock-0000000033. (2) Each client lists children of /locks/resource and checks if its node has the lowest sequence number. (3) If yes, the client holds the lock. (4) If not, the client watches the node with the next-lower sequence number and waits for its deletion event. (5) When the predecessor is deleted (holder released or crashed), the watcher is notified and rechecks. Ephemeral nodes are automatically deleted when the client's ZooKeeper session expires (session = heartbeat within session timeout). This provides automatic crash recovery without TTL races. The sequential ordering avoids thundering herd — only one client is notified per release.

Q8. What is split-brain in the context of distributed locks?

Split-brain occurs when two processes simultaneously believe they hold the same distributed lock. Causes: (1) Redlock TTL expiry during GC pause — original holder's lock expires, new holder acquires it, original holder resumes not knowing its lock expired. (2) Network partition — a partitioned Redis node still returns 'lock held' to a client while the majority of nodes have already re-granted the lock. (3) Redis leader failover with replication lag — lock acquired on master, master crashes before replicating, new master has no record of the lock, another client acquires. Mitigation: fencing tokens (storage enforces monotonicity). For strong guarantees, use ZooKeeper (CP system) instead of Redis (AP system).

Q9. How do PostgreSQL advisory locks differ from row-level locks?

PostgreSQL advisory locks are application-controlled locks not tied to specific tables or rows. They are identified by a user-supplied 64-bit integer key (you choose the semantics). `pg_advisory_lock(key)` — session-level, holds until `pg_advisory_unlock(key)` or session ends. `pg_advisory_xact_lock(key)` — transaction-level, auto-released at commit/rollback. `pg_try_advisory_lock(key)` — non-blocking, returns false if not available. Row-level locks (`SELECT FOR UPDATE`) lock specific rows and block concurrent `UPDATE`s. Advisory locks are more flexible — they can represent any shared resource concept, not just DB rows. They participate in the same transaction as business writes (useful for atomic lock-and-write semantics). Limitation: no TTL, so crash recovery depends on connection drop.

Q10. What happens when the Redis node holding a lock crashes?

Single Redis node: the lock data is lost. If the node restarts (not a persistent crash), it starts with an empty keyspace — the lock is effectively released. If using RDB/AOF persistence with `fsync`, the lock key may persist through a restart — the TTL will have partially expired. Best practice: use Redis with `appendonly yes` and `appendfsync always` for lock keys. With Redlock (N nodes): a single node crash is tolerated (N-1 nodes still form quorum). If the crashed node restarts after TTL seconds, its state is consistent with the lock having expired. Kleppmann's concern: if the node restarts before TTL expires and the lock was held, a new lock can be acquired on the restarted node, potentially reducing quorum count for a concurrent lock attempt.

Q11. How do you prevent a client from accidentally releasing another client's lock?

Use a unique random value (UUID) as the lock value and verify ownership before deletion. `SET lock:resource uuid NX EX 30` — the value is the client's UUID. On release: compare-and-delete atomically using a Lua script: `'if GET(key) == uuid then DEL(key)'`. Without this, Client A acquires lock, lock expires, Client B acquires same lock, Client A tries `DEL` (thinking it still holds it) — and deletes Client B's lock. The Lua script makes `GET + DEL` atomic. Redisson handles this automatically in `RLock.unlock()`, which internally uses a Lua script to verify the lock identity before releasing.

Q12. What are the use cases where you should NOT use a distributed lock?

Avoid distributed locks when: (1) You can use database-level serialization (`SELECT FOR UPDATE` or serializable transactions) — simpler and same transactional guarantees. (2) The operation is idempotent — retry without coordination is safe. (3) You need high throughput — distributed locks serialize operations; use partitioning (lock per user/resource) to increase parallelism rather than one global lock. (4) The 'locked' operation spans user interactions (e.g., user fills a form) — never hold a lock for seconds to minutes; use optimistic locking instead. (5) You need exact-once semantics for external systems (payments, emails) — distributed locks alone can't provide exactly-once due to split-brain; combine with idempotency keys.

Q13. How do you implement a try-lock with timeout and return pattern?

A try-lock with timeout polls or waits up to a deadline: in Redis, implement with a retry loop: call `SET NX EX`; if null, `Thread.sleep(backoff)` and retry until deadline. In Redisson: `lock.tryLock(waitTime, leaseTime, TimeUnit)`. In ZooKeeper via Curator: `mutex.acquire(timeout, unit)` returns boolean. The key design decisions: (1) polling interval (100ms-500ms for Redis; ZK uses watches for instant notification). (2) Max wait timeout (fail fast vs. queue up). (3) Whether to return false or throw an exception on timeout. Always prefer `tryLock` with timeout over indefinite `acquire()` to prevent threads from waiting forever on a crashed lock holder.

Q14. Explain lock lease renewal and why it is needed.

Lock lease renewal extends the lock TTL periodically while the holder is still active. Without renewal: a 30-second TTL means any operation taking more than 30 seconds loses the lock — a problem for long database queries, external API calls, or file operations. With watchdog (Redisson): a background thread renews the TTL every `TTL/3` seconds. The renewal is tied to JVM liveness — if the JVM crashes, the watchdog thread stops, the TTL is not renewed, and the lock expires within the original TTL. This balances: (1) Liveness — long-running work doesn't lose the lock. (2) Safety — crashes eventually release the lock.

Always size TTL to cover the expected operation duration plus safety margin; watchdog handles the rest.

Q15. What is the thundering herd problem with distributed locks and how do you solve it?

Thundering herd: when a lock is released, all waiting clients simultaneously receive the notification (or their polling interval fires) and attempt to acquire the lock. Only one succeeds; all others fail and immediately retry — causing a spike of Redis load and failed connections. Solutions: (1) Randomized backoff — each failed acquisition waits $\text{random}(\text{baseDelay} * \text{attempt})$ before retrying. (2) ZooKeeper sequential nodes — each waiter watches only the predecessor node; only one client is notified per release. (3) Queue-based — explicit work queue (Redis list, RabbitMQ) instead of a lock; one worker dequeues at a time. (4) Lock striping — partition the resource into N sub-locks so N clients can work concurrently, reducing contention by $N\times$.

Q16. How do you handle distributed lock failure in a microservices architecture?

Strategy: (1) Fail fast — if lock acquisition fails within timeout, return 503 Service Unavailable with Retry-After header; the client retries later. (2) Idempotency key — ensure the protected operation is idempotent so retries are safe. (3) Fallback — if the lock service (Redis) is unavailable, decide whether to fail open (proceed without lock — acceptable for non-critical resources) or fail closed (reject requests). (4) Circuit breaker — wrap lock acquisition in a circuit breaker; if Redis has high failure rate, open the circuit and route to fallback. (5) Observability — monitor lock acquisition latency, timeout rate, and queue depth. Lock contention is often a symptom of upstream bottlenecks.

Q17. What is the difference between a distributed lock and a distributed semaphore?

A lock allows exactly one holder at a time (binary semaphore). A distributed semaphore allows up to N concurrent holders. Use case: rate limiting — allow up to 10 concurrent database connections from all service instances. Redisson provides `RSemaphore`: `semaphore.acquire()` decrements a Redis counter; `semaphore.release()` increments it. The counter is initialized with N permits. Atomically decremented with a Lua script. If the counter reaches 0, new acquires block. Unlike a lock, multiple processes can proceed simultaneously up to the configured limit. Also useful for resource pooling (connection slots, API rate limits per IP).

Q18. How do you test distributed lock behavior in integration tests?

Testing distributed locks requires simulating concurrent contention. Approach: (1) Use an embedded Redis (e.g., testcontainers with Redis image) for integration tests. (2) Launch two threads, both trying to acquire the same lock. Assert only one succeeds. (3) Release the lock and assert the second thread can then acquire it. (4) Simulate TTL expiry: acquire with 1-second TTL, `Thread.sleep(1500ms)`, assert lock is now available. (5) Simulate crash: acquire lock, do NOT release, wait for TTL, verify another thread can acquire. (6) Test fencing token monotonicity: acquire twice, assert token is strictly increasing. (7) Inject Lua script failures via a mock Jedis and verify that partial acquisition is cleaned up properly.

Q19. What is a distributed lock anti-pattern and what should you replace it with?

Anti-pattern: long-held distributed locks for user-session-level operations. Example: lock acquired when a user opens an edit form, held until they submit (could be minutes). Problems: (1) Lock TTL makes it unreliable over minutes — either TTL must be huge (defeating crash safety) or watchdog is used (but browser close doesn't release it). (2) Locks serialise all editors, ruining UX. Replacement: optimistic locking — show the resource with its version number. When the user submits, check `If-Match` or `@Version`. If conflict, show 'someone else changed this document' with a merge UI. Only hold a short distributed lock (milliseconds to seconds) for atomic operations like payment processing, not for user think-time periods.

Q20. How does lock granularity affect system throughput?

Coarse-grained lock: one lock for all orders. Every order operation serialises — throughput = 1 operation at a time. Fine-grained lock: one lock per order ID. N orders can be processed concurrently — throughput scales

with N. Design principle: lock the smallest possible scope identifying the unique conflicting resource. Key strategies: (1) Lock per entity ID (lock:order:{orderId}). (2) Lock per user + resource type (lock:user:{userId};profile). (3) Lock per partition (lock:partition:{hash(orderId) % 256}) for bounded lock count. Fine-grained locks reduce contention but increase lock management overhead. The optimal granularity balances throughput with the acceptable number of concurrent locks in Redis/ZK.